Хэш функция на базе генератора случайных чисел

- *Анатолий М. Георгиевский* , ИТМО, 2026 (https://github.com/AnatolyGeorgievski)
- автор благодарит *ĀŁΞX* (https://github.com/Alex20129), за обсуждение принципов построения mwc-хэш и помощь в тестировании

> Представляем метод синтеза некриптографической хэш-функции на алгоритмах MWC128.

- Хэш функция на базе генератора случайных чисел
  - Алгоритм MWC128-hash
  - Алгоритм PRNG-hash
  - MXM - миксеры
  - MUM - миксеры
  - MWC - миксеры
  - RRX - миксер
  - RNG - миксер
  - MRC - миксеры
  - Векторизация цикла MWC-hash
  - Методика тестирования
  - Результаты работы

Известно множество некриптографических функций, которые могут быть использованы для индексации баз данных и проверки контрольных сумм массивов данных. К таким функция предъявляются требования - низкая вероятность коллизий. Традиционно эта ниша занята алгоритмами CRC64, однако линейное преобразование LCG или CRC не избавляет от коллизий. В сетевых протоколах и архиваторах используются различные некриптографические функции, среди которых выделяется xxHash64 (Yann Collet). Нас интересует разработка быстрой хэш-функции для работы потоке данных и с большими массивами данных, такими как тензорные данные языковых моделей. Алгоритм вычисления хэш должен допускать возможность считать параллельно с разбивкой данных на сегменты и векторизовать вычисления.

Современные хэш-функции строятся по принципу SPONGE-SQUEEZE: состоят из цикла впитывания данных и отжимания "губки". Мы предлагаем архитектуру хэш функции построенную на обратимых линейных и нелинейных операциях:

1. Линейный генератор с возможностью выполнять skip/jump и merge на потоке данных.

2. Функция пермутации-миксер выходных данных.

Обратимость операций и линейный генератор в основе обеспечивают возможность выполнения операций над потоками данных:

1. `next(s)` - генерация следующего состояния.
2. `prev(s)` - генерация предыдущего состояния без использования контрольных точек, на основе текущего состояния, обратный шаг генератора.
3. `skip(s,n)` - пропуск шагов генератора, n-элементов данных в потоке.
4. `merge(s1, s2, n)` - слияние хешей от двух сегментов, с учетом дистанции между сегментами
5. `jump(s, m)` пропуск сегмента на основе константы, например 2^m, поддерживается ограниченных набор сдвиговых констант.
6. `mix(x)` функция смешивания на выходе генератора
7. `unmix(x)` функция обратная миксеру

Всеми этими качествами обладает генератор на основе CRC или LFSR. Полным аналогом в модульной арифметике является MWC генератор, который представляет собой редуцирование после сдвига, аналогично шагу CRC алгоритма.

Тестирование хэш-функций выполняется в пакете SMHasher. Тесты ориентированы на выявление слабых бит в хэше и поиск коллизий.

В представленной работе мы ссылаемся на опыт разработки LXM генератора [4], где авторы использовали множество типовых миксеров для придания функции генератора ГПСЧ необходимого качества, для прохождения тестов без коллизий.

В ходе разработки алгоритма мы повторили результаты [4] с использованием генератора `Xoroshiro128p` И MXM-миксера выходных значений предложенного авторами в работе. Кроме того протестировали ряд других миксеров. Исходный код подготовлен для теста SMHasher см. xoroshiro-hash. Для сравнения реализовали собственную версию функции xxHash64 переносимую.

Мы видим две цели: получить качественный алгоритм PRNG-hash и разработать модель обратимого, дописываемого хэша. Первая цель включает подбор параметров и выходных миксеров для достижения высоких показателей в тестах. Вторая цель создание математического объекта способного заменить CRC в задачах контроля целостности и кэширования больших объемов данных, таких как тензоры языковых моделей.

Алгоритм MWC128-hash

```
uint64_t mwc128_hash(const uint8_t *data, uint64_t len, uint128_t seed) {
        uint128_t h = unmix(seed);
        for (int i=0; i<len>>3; i++){
                h+=*(uint64_t*)data; data+=8;
                h = (uint64_t )h*A1 + (h>>64); // итерация MWC128
        }
        if (len&7) {
                int r = len&7;// reminder
                uint64_t d = 0;
                memcpy(&d, data, r); data+=r;
                h+= (d);
                h = ((uint64_t )h<<(64-r*8))*A1 + (h>>(r*8));
        }
        return mix(h);
}
```

Параметры алгоритма:

- `A1` - параметр генератора MWC128, где $P = A_1 \cdot 2^{64} - 1$ простое число.

В составе алгоритма используется обратимая функция миксера: `mix` и `unmix` .

В составе алгоритма может использоваться миксер Doug Lea's. Это не самый сильный миксер, но самый простой и оптимально подошел к нашей задаче.

```
// Doug Lea's MXMX- mixing function
static inline uint64_t mix_lea(uint64_t h) {
  h ^= h >> 32;
  h *= 0xdaba0b6eb09322e3ull;
  h ^= h >> 32;
  h *= 0xdaba0b6eb09322e3ull;
  h ^= h >> 32;
  return h;
}
```

Ниже мы приводим множество миксеров, с которыми можно комбинировать выход функции генератора. Миксеры можно выбирать разной сложности в архитектуре XMXMX или более простые: X, MX, XMX. Прохождение тестов SMHasher обеспечивают миксеры XMXMX.

Выбор констант генератора должен давать равномерное распределение бит в тестах SMHasher и TestU01. Среди тестов TestU01 мы выделяем тест `Linear-complexity` с существенно большим периодом. Каждый бит тестируется отдельно.

```
for (int i=32; i-->0;){
        scomp_LinearComp(&gen, NULL, 1, 400000, i, 1);
}
```

Алгоритм PRNG-hash

Алгоритм построен на генераторе Xoroshiro128, как в работе [4], LXM. Применяется рандомизация и миксер выходных значений. В качестве миксера выбрана функция `mix_lea()` , Doug Lea's mixing function.
Для построения хэш функции использован генератор `Xoroshiro128+` , выходным значением генератора является `plus` скрамблер от внутреннего состояния.

```
uint64_t xoroshiro_hash(const uint8_t *data, uint64_t len, uint64_t* seed) {
        uint64_t s[STATE_SZ];
        unmix(s, seed);
        for (int i=0; i<len; i++) {
                s[0] ^= *data++;
                next(s);
        }
        return mix(s);
}
```

Функция миксера, как и функция генератора - обратима. Это свойство может быть использовано при контроле целостности в операции дописывания данных в конец файла.

Генераторы xorshift

1. $x \leftarrow x \oplus (x \gg a)$
2. $x \leftarrow x \oplus (x \ll b)$
3. $x \leftarrow x \oplus (x \gg c)$

-- Классический генератор, в котором можно комбинировать левые и правые сдвиги (a,b,c).

Таблица вариантов алгоритма

Вариант	С-код (порядок операций)
A_0	`x ^= x << a; x ^= x >> b; x ^= x << c;`
A_1	`x ^= x << c; x ^= x << b; x ^= x >> c;`
A_2	`x ^= x << c; x ^= x >> b; x ^= x << a;`
A_3	`x ^= x >> c; x ^= x << b; x ^= x >> a;`
A_4	`x ^= x << a; x ^= x << c; x ^= x >> b;`
A_5	`x ^= x >> a; x ^= x << c; x ^= x << b;`
A_6	`x ^= x >> b; x ^= x << a; x ^= x << c;`
A_7	`x ^= x << b; x ^= x >> a; x ^= x >> c;`

Операции

[1402.6246] An experimental exploration of Marsaglia's xorshift generators, scrambled
Sebastiano Vigna
ACM Transactions on Mathematical Software, Volume 42, Issue 4, Article No. 30 (2016)
PDF: https://vigna.di.unimi.it/ftp/papers/xorshift.pdf, arXiv:1402.6246

[1404.0390] Further scramblings of Marsaglia's xorshift generators, Sebastiano Vigna
arXiv:1404.0390 (2014)
PDF: https://vigna.di.unimi.it/ftp/papers/xorshiftplus.pdf

Варианты алгоритма с задержкой

1. $t = x_1 \oplus (x_1 H_1 a)$
2. $t = t \oplus (t H_2 b)$
3. $x_i = x_{i+1}; \quad i = 1, 2, ...r - 1$
4. $x_r = (x_r \oplus (x_r H_3 c)) \oplus t$

-- Обобщенная модель генератора xorshift с задержкой, где H_1, H_2, H_3 операции сдвига влево или вправо. Для увеличения состояния используется перестановка элементов вектора состояния.
[111] G. Marsaglia. Xorshift RNGs. Journal of Statistical Software, 8(14):1-6, 2003.

Генераторы xoroshiro128

```
static inline void xoroshiro128_next(uint64_t* s) {
        uint64_t s0 = s[0];
        uint64_t s1 = s[1];
        s1 ^= s0;
        s0 = rotl(s0, A);
        s[0] = s0 ^ s1 ^ (s1 << B);
        s[1] = rotl(s1, C);
}
```

Известные константы сдвигов [A,B,C]: .

В исходном коде определены три функции xoroshiro с состоянием 128 бит.

Генераторы PRNG на базе ARX-структур

За основу берется четверть раунда ChaCha20 или Salsa20, минимальное число раундов 8.

```
static inline void half_qr(uint32_t s[4]) {
    uint32_t a = s[0], b = s[1], c = s[2], d = s[3];

    a += b; d = ROTL32(d^a, R1);
    c += d; b = ROTL32(b^c, R2);

    // перестановка — один из лучших вариантов {3,2,0,1}
    uint32_t t[4] = {a, b, c, d};
    s[0] = t[1];    // b
    s[1] = t[3];    // d
    s[2] = t[0];    // a
    s[3] = t[2];    // c
}
```

В результате получается алгоритм подобный SipHash

MXM - миксеры

Отправной точкой исследования является вариант финального миксера из функции `MurmurHash3` . Кроме того, в составе разных хэш-функций мы нашли много вариантов миксера со структурой XMXMX:

1. $x \leftarrow x \oplus (x \gg a)$
2. $x \leftarrow x \cdot Prime_1$

3. $x \leftarrow x \oplus (x \gg b)$
4. $x \leftarrow x \cdot Prime_2$
5. $x \leftarrow x \oplus (x \gg c)$

Подобный миксер `Doug Lea's` использован в работе [4], `SplitMix64` предлагается авторами в качестве рандомизатора SEED при инициализации `Xoroshiro128` и генераторов с большой разрядностью состояния. `Avalanche` миксер заимствован из функции `xxHash64` . Сложность миксера может варьироваться от одной итерации Mul-Xor-shift до двух-трех. Мы рассматриваем миксеры со структурой: MX, XMX, MXMX, XMXMX.

MurmurHash3 64-bit avalanche mixer представлен в проекте SMHasher, Appleby 2016.

Mixer	a	Prime1	b	Prime2	c
MurmurHash3	33	0xff51afd7ed558ccd	33	0xc4ceb9fe1a85ec53	33
Doug Lea's	32	0xdaba0b6eb09322e3	32	0xdaba0b6eb09322e3	32
SplitMix64	30	0xbf58476d1ce4e5b9	27	0x94d049bb133111eb	31
Avalanche	33	0xC2B2AE3D27D4EB4F	29	0x165667B19E3779F9	32

David Stafford's improved mixing functions опубликованы в блоге автора.

Теблица David Stafford's Mixer

Mixer	a	Prime1	b	Prime2	c	max ord
Mix01	31	0x7fb5d329728ea185	27	0x81dadef4bc2dd44d	33	★★
Mix02	33	0x64dd81482cbd31d7	31	0xe36aa5c613612997	31	☆☆
Mix03	31	0x99bcf6822b23ca35	30	0x14020a57acced8b7	33	★☆
Mix04	33	0x62a9d9ed799705f5	28	0xcb24d0a5c88c35b3	32	★★
Mix05	31	0x79c135c1674b9add	29	0x54c77c86f6913e45	30	★★
Mix06	31	0x69b0bc90bd9a8c49	27	0x3d5e661a2a77868d	30	☆★
Mix07	30	0x16a6ac37883af045	26	0xcc9c31a4274686a5	32	★★
Mix08	30	0x294aa62849912f0b	28	0x0a9ba9c8a5b15117	31	★☆
Mix09	32	0x4cd6944c5cc20b6d	29	0xfc12c5b19d3259e9	32	★☆

Mixer	a	Prime1	b	Prime2	c	max ord
Mix10	30	0xe4c7e495f4c683f5	32	0xfda871baea35a293	33	★★
Mix11	27	0x97d461a8b11570d9	28	0x02271eb7c6c4cd6b	32	☆★
Mix12	29	0x3cd0eb9d47532dfb	26	0x63660277528772bb	33	★★
Mix13	30	0xbf58476d1ce4e5b9	27	0x94d049bb133111eb	31	☆★
Mix14	30	0x4be98134a5976fd3	29	0x3bc0993a5ad19a13	31	★★

Любой из представленных миксеров дает хороший лавинный эффект улучшает результат прохождения теста SMHasher.

Выбор констант миксера

Уменьшение коллизий основано на том что число - множитель - простое число и является генератором мультипликативной группы $Z/2^{64}Z$, т.е возведение в степень по модулю 2^{64} дает максимальную длину последовательности. Если рассматривать миксер, как обратимую операцию, то его логично строить с использованием сопряженных чисел $P_2 = P_1^{-1} \bmod 2^{64}$. Таблица миксеров не содержит таких сочетаний. Так что идея комбинировать простые числа и обратные - оригинальная и может использоваться для выбора множителей. Критерием выбора константы является прохождение теста SAC и BIC. Кроме того, хорошим критерием является статистическое распределение единиц и ноликов по разрядам числа, тест `difficulty/hashrate` , который мы предлагаем использовать для оценки качества скрамблера/миксера. Тест позволяет получить статистику распределения бит по битовой сложности.

Обратимость операции Mixer

Операции вида $h = h \oplus (h \gg m)$ обратимы, операции с $m \geq 32$ обратны сами себе.

```
uint64_t reverse_xor_shift(uint64_t h, unsigned int m) {
    h ^= (h >> m);
    if (2*m < 64) h ^= (h >> (2*m));
    if (4*m < 64) h ^= (h >> (4*m));
//      . . .
    return h;
}
```

Для сдвигов $m = 26...31$ достаточно двух действий.

Таблица Обратимость xor-shift

m	обратная операция
33	`h ^= h >> 33;` (обратная себе)
32	`h ^= h >> 32;` (обратная себе)
31	`h ^= h >> 31; h ^= h >> 62;`
30	`h ^= h >> 30; h ^= h >> 60;`
29	`h ^= h >> 29; h ^= h >> 58;`
28	`h ^= h >> 28; h ^= h >> 56;`
27	`h ^= h >> 27; h ^= h >> 54;`
26	`h ^= h >> 26; h ^= h >> 52;`

Таблица Обратных чисел для операции unMix

Mixer	a	$Prime_1^{-1}$	b	$Prime_2^{-1}$	c
unMix_Lea		`0xa6f8e26927e132cb`		`0xA6F8E26927E132CB`	
unSplitMix64		`0x96de1b173f119089`		`0x319642B2D24D8EC3`	
unAvalanche		`0x0ba79078168d4baf`		`0xE9E9F4C41D6DF849`	
unMix01		`0x4c5ff4596f4a2f4d`		`0x4D6DFF26C61D8485`	
unMix02		`0xfaa6b01ec53551e7`		`0x9BB5680ABE73E627`	
unMix03		`0xcb94d79668acb81d`		`0xB0E38339F3478507`	
unMix04		`0x8e7fc80bbd7bbe5d`		`0x13A10FC6E8A1817B`	
unMix05		`0x4d7dac66e4190d75`		`0xABCDFD8F7FB3248D`	
unMix06		`0x7b3e9e7a952f25f9`		`0x059575CCED6AAC45`	
unMix07		`0x2047f2bc3066a28d`		`0x74CA595AE625F12D`	
unMix08		`0x294dde0da6c4a4a3`		`0xDD04785DF6D8F6A7`	
unMix09		`0x4434dd7ecb5ab665`		`0x8BFD21AC23740E59`	
unMix10		`0x49e0439cd61fd05d`		`0x02054AEE6574CB9B`	

Mixer	a	$Prime_1^{-1}$	b	$Prime_2^{-1}$	c
unMix11		`0xf542db7fa2580f69`		`0x1D16CF44AFE4F743`	
unMix12		`0x66d6694153c4d533`		`0xCE487C2C5BA60273`	
unMix13		`0x96de1b173f119089`		`0x319642B2D24D8EC3`	
unMix14		`0x4ab3236cb05fc05b`		`0xAB56D1249120401B`	

Для обращения операции умножения `*=` надо найти обратное число $B = A^{-1}$ такое, что

$$A \cdot B \bmod 2^{64} \equiv 1$$

Таким образом функция XMXMX- avalanche Mixer является обратимой функцией.

MUM - миксеры

MUM-миксер (MUltiply and Mix) - это просто операция умножения 64 x 64 бит с результатом 128 бит.

1. $\{r_{lo}, r_{hi}\} = A \cdot B$
2. $r_{lo} \oplus r_{hi} \bmod 2^{64}$

Вместо того чтобы описывать свойства миксера, я представлю класс генераторов случайных чисел. Он начинается с генератора Lehmer64 и WYrand.

```
static uint128_t state=SEED;
uint64_t lehmer64() {
    state *= 0xda942042e4dd58b5ULL;   // фиксированный множитель
    return (uint64_t)(state >> 64);   // берём старшие 64 бита как результат
}
```

Множитель выбирается так, чтобы период повтора последовательности был максимальный. Максимальный период генератора 2^{126}. Как выбрать подобный множитель - случайно. Случайное число должно быть генератором мультипликативной группы и давать хороший лавинный эффект. Если число дает хороших лавинный эффект, то будет проходить тесты PRNG.

```
uint64_t seed = SEED;
uint64_t wyrand() {
    seed += UINT_64_C(0x60bee2bee120fc15);
    uint128_t y = seed * (uint128_t)UINT64_C(0xa3b195354a39b70d);
    return y ^ (y >> 64);
}
```

Если из операции можно сделать трансформацию с большим периодом повтора, то это хороший кандидат на изготовление генератора последовательности псевдослучайных чисел. Некоторые числа обладают свойством генератора последовательности на операции сложения, Weyl-подобные числа. Некоторые - являются генератором поля $Z/2^{64}Z$ по умножению. От множителя требуется, чтобы он давал хороший лавинный эффект с вероятностью 50% по каждому биту. Для хорошего нелинейного смешивания можно использовать возведение в степень простого числа - генератора мультипликативной группы.

В своем проекте, мы использовали трансформацию mum в режиме CTR-генератора (counter mode) для исследования статистических свойств констант MUM-миксера:

```
uint64_t mum_mixer() {
    seed += gamma; // Любая Weyl-константа, 1 или -1
    uint128_t y = mum(seed, Prime);
    return y ^ (y >> 64);
}
```

В тесте инкремент `gamma` выбирается так, чтобы суммирование давало перебор всех значений по модулю 2^{64}. Обычно для этих целей применяют Weyl-подобные константы, включая `1` и `-1`. Наиболее распространенной является константа *золотое сечение*.

Есть ряд проектов, которые применяют операцию `mum` (Multiply and Mix) в ядре хэш-функции:

- wyhash
- rapidhash
- mumhash

Операция строится по принципу

- `state ^= mum(data ^Prime1, Prime2);`
- `state ^= mum(data1^Prime1, data2^Prime2);`

Последний вариант обладает наивысшей возможной производительностью, поскольку на одно умножение используется два элемента данных. Хотя, можно рассмотреть вариант:

- `state ^= mum(data1^Prime1, Prime2) ^ data2;`

Возможно, чуть лучше с простыми числами и коллизиями обстоит дело в проекте Владимира Макарова MUM-hash. Там операция `mum` использует сложение $r_{lo} + r_{hi} \bmod 2^{64}$. В проекте представлен свой генератор ГПСЧ на операции MUM с запаздыванием.

```
#define MUM_SZ 2 // размер состояния 2-16:128 ... 1024 бит
void mum_prng_update(uint64_t* state) {
    for (int i = 0; i < MUM_SZ - 1; i++)
        state[i] ^= mum (state[i + 1], primes[i]);
    state[MUM_SZ - 1] ^= mum(state[0], primes[MUM_SZ - 1]);
}
```

Иногда операцию MUM хочется назвать Multyply and Rotate (MUR), тогда получаем базовую идею MurMur-хэша. В нашем случае правильно было назвать *Multiply and Reduce*, понимая под редуцированием - операцию редуцирования по модулю.

Мои тесты показали, что на операции MUM миксеры ведут себя лучше, чем XMXMX-миксеры. Операция MUM быстрее XMXMX на 64 битных процессорах. Чтобы сделать осознанный выбор мы тестировали и подбирали множители с разными Weyl-константами.

Обратимость операции MUM

Умножение можно разложить, как сумму операций сдвиг-сложение с последующим редуцированием. Если операция сложения выполняется по модулю $P = 2^{64} - 1$

Операции сдвига влево

$$\{r_{lo}, r_{hi}\} = x \cdot 2^n \bmod 2^{64} - 1$$

Эквивалентна операции `ROTL64(x, n)`

Операция `rlo + rhi` может быть интерпретирована как Lazy Reduction по модулю $2^{64} + 1$.

Использование Lazy Reduction делает эти примитивы непригодными для криптографии (например, для MAC или шифрования), так как отсутствие полного приведения по модулю может создавать неочевидные алгебраические уязвимости и коллизии. Чтобы получить точное и обратимое значение, нужно результат операции корректировать.

Сравнение схем редукции MUM (MUltiply-adn-Mix)

Операция	Формула	Интерпретация модуля	Обратимость	Производительность
MUM (XOR)	`rlo ^ rhi`	GF(2)/$(x^{64}+1)$	Условная	⭐⭐⭐⭐⭐
MUM (ADD)	`rlo + rhi`	Lazy $2^{64}-1$	Условная	⭐⭐⭐⭐
MUM (SUB)	`rlo - rhi`	Lazy $2^{64}+1$	Условная	⭐⭐⭐⭐
MOD (pseudo-Mersenne)	`rlo + M·rhi`	$2^{64}-M$	Полная	⭐⭐⭐

Примеры реализации

```
// XOR-редукция GF(2)/($x^{64} + 1)
uint64_t mum_xor(uint64_t a, uint64_t b) {
    __uint128_t p = (__uint128_t)a * b;
    return (uint64_t)p ^ (uint64_t)(p >> 64);
}
// ADD-редукция (Lazy reduction mod 2^64-1)
uint64_t mum_add(uint64_t a, uint64_t b) {
    __uint128_t p = (__uint128_t)a * b;
    return (uint64_t)p + (uint64_t)(p >> 64);
}
```

MWC - миксеры

Наиболее общее определение операции MWC можно ввести через операцию Montgomery-friendly reduction, когда одновременно выполняется выполняется сдвиг на слово и редуцирование по модулю. Операция модульного умножения Montgomery-friendly будет давать результат
$r = A \cdot B \cdot 2^{-n} \bmod P.$

Предлагаю рассматривать операцию сдвига и редуцирования MWC в качестве миксера, наравне с функцией MUM-миксера. В нашей постановке задачи мы все время говорим об математически точном преобразовании, в то время как константы в MUM миксерах выбираются практически случайным образом по критерию достижения лавинного эффекта.

```
h = ((h>>64) + (uint64_t)h * A1) * MC;
```

Константы `MC` могут выбираться для Avalanche Effect, для достижения наилучшего лавинного эффекта. При этом операция MWC миксера - обратима. Вопрос ставится: возможно ли синтезировать на сдвиговой операции MWC128 быстрый и качественный алгоритм хеша, используя константы найденные случайным образом по критериям MUM-миксера. Так мы решили модифицировать функцию FNV-1a и вместо операции умножения `h = (h^data) * MC` запишем `MUM(h^data, MC)` или `MWC(h^data, MC)` или операцию умножения с последующим редуцированием по модулю простого числа, например $2^{64} - 2^s + 1$. Модульная операция - редуцирование по модулю простого числа может работать, как миксер.

Представим несколько вариантов миксеров MWC.

Введем обозначения: $[X]^k = \left\lfloor \frac{X}{2^k} \right\rfloor$, $[X]_k = X \bmod 2^k$

Представляем варианты миксера на шаге редуцирования различных простых чисел [Plantard]:

$$
\begin{aligned}
&h \leftarrow [h]_n + [h]^n && \text{Mersenne} && P = 2^n - 1 && (1)\\
&h \leftarrow [h]_n + [h]^n 2^s \mp [h]^n && \text{generalized Mersenne} && P = 2^n - 2^s \pm 1 && (2)\\
&h \leftarrow [h]_n + [h]^n \cdot M && \text{pseudo Mersenne} && P = 2^n - M, M < 2^{n/2} && (3)\\
&h \leftarrow [h]^n + [h]_n \cdot M && \text{MWC, Rizel} && P = M \cdot 2^n - 1, M < 2^n && (4)\\
&h \leftarrow [h]^n - [h]_n \cdot M && \text{NTT, Proth} && P = M \cdot 2^n + 1, M < 2^n && (5)
\end{aligned}
$$

Заметим, что шаг миксера для *NTT-friendly* простых чисел выполняется над числами со знаком, в то время как *MWC* (Простые числа Ризеля) на числах без знака. В коде *NTT-friendly* итерация обозначается, как *signed MWC*. В литературе редукция MWC обозначается, как *Montgomery-friendly*, совмещает сдвиг и редуцирование по модулю простого числа.

В проекте мы тестируем миксеры: *MWC*, *NTT-friendly (signed MWC)*, *Mersenne*.

Операцию хэша можно представить в виде полинома, в котором участвует константа смешивания α.

$$
\begin{aligned}
&h = d_0 + d_1\beta + d_2\beta^2 + ... + d_{k-1} \cdot \beta^{k-1} && (6)\\
&h = d_0\alpha^{k-1} + d_1\alpha^{k-2} + ... + d_{k-2}\alpha + ... + d_{k-1} \quad \bmod P && (7)
\end{aligned}
$$

Операции сложения и умножения можно выполнять в арифметике GF(2), с заменой операции сложения на XOR, а умножения - на полиномиальное умножение (CLMUL, умножение без переносов).

Для алгоритма MWC-hash мы выбираем $\alpha\beta \equiv 1 \bmod P$.

Величина α должна обладать некоторым свойством - максимальный период мультипликативной группы по модулю P. Для простых чисел Прота, NTT-friendly, максимальный период может быть найден как $\alpha^{(P-1)/w} \equiv 1 \bmod P$. Для чисел Ризеля, максимальный период при $w = 2$. По модулю $P = 2^{64}$ максимальный период 2^{62}.

В обобщенном виде итерацию с произвольным шагом можно представить одним из вариантов:

$$h \leftarrow (h \oplus d_i) M_c \mod P \quad \text{– шаг FNV-1a} \tag{8}$$
$$h \leftarrow h \cdot M_c \oplus d_i \mod P \quad \text{– шаг FNV-1} \tag{9}$$
$$h \leftarrow ([h]_n \cdot M_1 + [h]^n \cdot M_2) \oplus d_i \quad \text{– шаг MWC} \tag{10}$$
$$h \leftarrow ([h]_n \cdot 2^{64-n} + [h]^n) \cdot M_c \oplus d_i \quad \text{– шаг MWC} \tag{11}$$

Предложения по модификации функции FNV-1a(Glenn Fowler, Landon Curt Noll, and Kiem-Phong Vo) сводится к замене константы на простое число с хорошим лавинным эффектом, и замене модуля $P = 2^{64}$ на операцию редуцирования по модулю простого числа, после умножения $64 \times 64 \rightarrow 128$ бит. Помимо модульной операции, можно примерять различные операции смешивания типа MUM.

Интересные частные случаи миксера для генерализации MUM, multiply and mix:

- `h = rlo ^ rhi` -- *Multiply and mix*,
- `h = rlo + rhi` -- *Multiply with carry mix*,
- `h = rlo - rhi` -- *signed Multiply with borrow mix*
- `h = rlo - rhi + (rhi<<S)` -- *NTT-friendly*; $P = 2^{64} - 2^s + 1$
- `h = rlo ^ rhi ^ (rhi<<S)`
- `h = rlo ^ rhi ^ (rhi>>S)` -- *MXM*, *multiply with Xor mixer*,
- `h = rlo ^ ROTL(rhi, R)` -- *MRX*, *multiply with Rotate-Xor mixer*,
- `h = ROTL(rlo ^ rhi, R)` -- *MXR*, *multiply with Xor-Rotate mixer*,
- `h =(rlo + rhi) ^ ROTL(rhi, R)` -- *ARX*, *Add-Rotate-Xor mixer*,

Эксперименты с этими миксерами представлены в виде вариантов хэша FNV-1a.

Заметим, непременным условием хорошего миксера является возможность построения на итерации генератора последовательности псевдо-случайных чисел с максимальным периодом. Критерием хорошего миксера является простота числа `M` и баланс нулей и единиц. Множитель `M` должен обладать максимальным порядком мультипликативной группы $Z/2^{64}Z$. Дополнительные требования на качество констант `Linear Complexity` и прохождение тестов `Bit Independence Criteria` на кратных периодах 2^n, $n > 28$.

Генератор случайных чисел с выходом n-бит ($y = [s]_n$) на MWC128 - миксере:

$$d_i \leftarrow d_{i-1} + \gamma \tag{12}$$
$$s \leftarrow ([s]_n \cdot M_1 + [s]^n) \cdot M_c \oplus d_i \tag{13}$$

где γ - Weyl- подобная аддитивная константа, опция. Для достижения максимального периода, константы M1 и Mc должны быть взаимно простыми. Базовая константа M_c для Avalanche эффекта может выбираться, как простое число с максимальный порядком мультипликативной группы 2^{62} и балансом нулей и единиц. Максимальный порядок итерации MWC128 $\approx 2^{127}$.

RRX - миксер

Мы хотим рассмотреть обратимый миксер на операциях вращения ROR и XOR. Множественные операции вращения ROR можно представить как умножение на матрицу аффинного преобразования в поле GF(2). Операция ROR порождает матрицу - циркулянт, в которой все строки сдвинуты. Матрица полностью определена первой строкой или образующим полиномом p(x) в первой строке.

Произведение матрицы на вектор можно заменить на операцию полиномиального умножения образующего полинома с последующим редуцированием:

$$(a(x) \odot p(x)) \pmod{x^{64} + 1}$$

Это делает класс RRX-миксеров похожим на MUM, но с заменой операций умножения на полиномиальное умножение, умножение без переносов. Так же как в случае MUM-миксера мы будем искать константу (полином) для умножения, которая дает хороших эффект смешивания во всех разрядах.

Для начала мы можем рассмотреть скрамблер вида

$$x \mapsto x \oplus (x \lll i) \oplus (x \lll j)$$

и набор скрамблеров Sum0, Sum1 из стандарта SHA-256/-512.

Далее можно предложить общий метод представления подобных скрамблеров с произвольным числом операций ROR. Каждое вращение будет обозначено одним битом в образующем полиноме. Полином получается методом подставить единицу в функцию RRX-миксера, если она представлена операциями ROR.

Метод обращения RRX

Операция RRX-миксера порождает матрицу - циркулянт. Обратная циркуляционная матрица тоже циркулянт.

Циркулянт размера $n \times n$ полностью определяется первой строкой (или первым столбцом): $c = [c_0, c_1, c_2, \ldots, c_{n-1}]$
Матрица соответствует полиному

$$p(x) = c_0 + c_1 x + c_2 x^2 + \ldots + c_{n-1} x^{n-1}$$

Обратная матрица тоже циркулянт, соответствует полиному q(x) в кольце полиномов:

$$p(x) \cdot q(x) \equiv 1 \pmod{x^n - 1}$$

Это справедливо для любой арифметики, в т.ч. арифметики Галуа. При переходе к арифметике Галуа, операцию `-` заменяем на `+` и коэффициенты полинома представляем в поле GF(2).

Чтобы найти обратное преобразование надо найти обратный полином в кольце полиномов, $q = p^{-1}$:

$$p(x) \odot q(x) \equiv 1 \pmod{x^n + 1}$$

Классический метод нахождения обратного полинома основан на обобщенном алгоритме Евклида. Но мы предлагаем более простой способ. Нужно возводить полином p(x) в степень пока не будет найден обратный. Степень может варьироваться от 1 до n-1.

Для возведения полиномов в степень нам понадобится операция умножения полиномов с редуцированием `pmull` и алгоритм возведения в степень `p_pow` на её основе. Используется бинарное возведение в степень (exponentiation by squaring).

```
for (ex = 1; ex<64; ex+=2) {
        p_inv = p_pow(p, ex);
        if ( pmull(p, p_inv) == 1) break;
}
```

Не все полиномы обратимы. При выборе обратимого полинома следует использовать: нечётное число ненулевых коэффициентов (вес Хэмминга нечётный). Отсуствие общих делителей $GCD(p(x), x^n + 1) == 1$, тоже верный признак. Но проверять GCD() - избыточно, поскольку предлагаемый алгоритм поиска, основанный на возведении в степень, в противном случае не даст результата. Алгоритм можно оптимизировать чтобы проверять степени $2^e - 1$.

Если полином выбирается, как случайное число, то фильтр
`popcnt(p)&1==1` -- (нечетность числа коэффициентов), самый быстрый способ отсева

кандидатов.

Полином p(x) обратим ⇔ существует q(x) такое, что $p(x) \cdot q(x) \equiv 1 \pmod{x^n + 1}$
$\Leftrightarrow gcd(p(x), x^n + 1) = 1$

При синтезе произвольного скрамблера можно использовать критерий аналогичный MUM: баланс единиц и ноликов в каждом байте.

Произвольный скрамблер выражается в две инструкции и полностью описывается набором коэффициентов полинома p(x), битами числа p;

```
uint64_t scrambler(uint64_t x, uint64_t p){
  return pmull(x, p);// carry-less mul & XOR
}
```

Для хорошего обратимого скрамблера найдется обратное число `p_inv` , которое задает обратное преобразование.

реализация метода синтеза и обращения функции скрамблера на полиномиальном умножении см. pmull_mixer.c

RNG - миксер

Шаг алгоритма на произвольном генераторе ГПСЧ можно синтезировать следующим образом

$$s \leftarrow (rng(s) \oplus d_i) \cdot M_c \tag{14}$$
$$s \leftarrow rng(s) \oplus d_i \cdot M_c \tag{15}$$
$$s \leftarrow rng(s) \cdot M_c \oplus d_i \tag{16}$$

где `rng()` - шаг выбранного алгоритма ГПСЧ. И конечно мы решили опробовать методы FNV-1a с несколькими генераторами, в частности Xorshift и Xoroshiro.

Генератор случайных чисел можно векторизовать за счет ротации состояния однотипных RNG генераторов, как это предложено в проекте MUM-hash, где под операцией MUM понимается умножение $64 \times 64 \rightarrow 128$ с редуцированием в 64 бит.

```
#define MUM_SZ 2 // размер состояния 1-16:64 ... 1024
void mum_rng(uint64_t* state) {
    for (int i = 0; i < MUM_SZ - 1; i++)
        state[i] ^= mum (rng(state[i + 1]), Primes[i]);
    state[MUM_SZ - 1] ^= mum(rng(state[0]), Primes[MUM_SZ - 1]);
}
```

MRC - миксеры

Давайте представим, что умножение на большую константу может быть хорошим миксером. Идея MRC-миксера происходит из китайской теоремы об остатках (CRT) из алгоритма RNS to MRC- mixed radix conversion, преобразование из системы остаточных классов в позиционную систему. Пусть a1, a2 независимо посчитанные числа от входного потока данных по двум простым модулям p1, p2.

```
uint128_t mrc_mix(uint64_t a1, uint64_t a2) {
    const uint64_t y1 = mod_inverse(p2 % p1, p1);
    const uint64_t y2 = mod_inverse(p1 % p2, p2);
    return a1 * p2 * y1 + a2 * p1 * y2;
}
```

Подобный миксер можно использовать для увеличения разрядности MWC64-hash, при этом два (и более) модулей MWC могут считаться параллельно.

- см. Residue Number System (RNS) -- система остаточных классов
- см. Numeric Theoretic Transform (NTT) -- еще один метод смешивания, подобный дискретному Фурье преобразованию (DFT).

Обе трансформации - обратимы. Метод увеличения разрядности будем называть RNS-MWC-hash. Метод смешивания *бабочками* будем называть NTT-hash. Оба метода пригодны для реализации на GPU.

Векторизация цикла MWC-hash

Основная операция векторного хеш называется *folding* ("схлопывание"). В модульной арифметике сдвиг влево или вправо можно представить через умножение на сдвиговую константу $J_n = \alpha^n \bmod P$. Основание сдвиговой константы для простоты понимания можно принять 2. Но на практике основание α - любое число по модулю, которое является

генератором мультипликативной группы с максимальным периодом. Я бы предложил выбирать основание таким образом, чтобы сдвиговые константы одновременно удовлетворяли двум критериям: генератор мультипликативной группы по модулю P и максимальный порядок по умножению в кольце Z/2^64Z. Качество сдвиговой константы в качестве миксера можно оценивать по балансу единичных и нулевых бит HammingWeight и тесту Bit Independence, который можно применить отдельно к операции *folding*.

Результат умножения - это два числа (R_{lo}, R_{hi}) - старшая и младшая части числа. Чтобы сдвинуть эти два числа на $n = 64 \cdot k$ бит, выполняется операция *folding*(n):

$$Fold(R_{lo}, R_{hi}, n) : (R_{lo} + R_{hi}2^w) \leftarrow R_{lo} \cdot J_n + R_{hi} \cdot J_{n-w}$$

Эта операция выполняется над векторами $n = 2k \cdot w$- элементов и не требует редуцирования по модулю. Операция редуцирования по модулю выполняется после завершения цикла и после выравнивания сдвигов элементов вектора, т.н. отложенная операция редуцирования. Операцию сложения с выравниванием сдвигов назовем *merge*. Именно такой подход мы предлагаем использовать на GPU для параллельного расчета хеш-функции от тензорных данных, векторизация выполняется по warp-группе, 32 элементов.

Ускорение на CPU с векторами 512 бит достигает максимума также при длине сдвига в операции *folding* порядка 2048 бит, т.е. с шагом 32 элемента. При этом может быть использована векторная арифметика системы команд AVX-IFMA, целочисленные векторные операции Fused-multiply-accumulate.

Операция *folding* требует тщательной оптимизации и использования ассемблерных вставок для оптимизации умножения, в проекте мы представляем переносимый (*portable*) вариант алгоритма с явной векторизацией цикла.

Приведем пример построения векторного алгоритма для разрядности 64-бит под использование модулей:

- $P = A_0 2^{32} \pm 1$ *MWC64* или *signed MWC64* (NTT-friendly) модули
- $P = 2^{64} - 2^{32} \pm 1$(Goldilocks and Three Bears) - используется в пост-квантовой криптографии и ZKP

Листинг ядра параллельного расчета хеш функции (псевдо-код), fold x4:

```
// вычисление сегмента
void hash_segment(uint8_t* data, size_t len, uint128_t* out){
        uint128_t h[4] = {0};
        // параллельно в группе x4
        while (len >= 32) {
                h[0] = fold(h[0] ^ GET_U64(data, 0), 32);
                h[1] = fold(h[1] ^ GET_U64(data, 8), 32);
                h[2] = fold(h[2] ^ GET_U64(data,16), 32);
                h[3] = fold(h[3] ^ GET_U64(data,24), 32);
                data+=32, len-=32;
        }
        // слияние по группе (merge)
        h[0] = fold(h[0], 16) + h[2];
        h[1] = fold(h[1], 16) + h[3];
        h[0] = fold(h[0],  8) + h[1];
        *out = h[0];
}
```

В ядре не применяется модульная операция, можно использовать lazy-reduction - отложенную операцию модульного редуцирования.

Листинг процедуры расчета хеш функции (псевдо-код):

```
size_t n_segments = data_len / SEGMENT_SIZE;
uint128_t segment[n_segments];
for (size_t k=0;k<n_segments; k++) {
        hash_segment(data, SEGMENT_SIZE, &segment[k]);
        data+= SEGMENT_SIZE;
}
uint64_t h = seed;
// слияние по сегментам  (merge)
for (size_t k=0;k<n_segments; k++) {
        h = fold(h, SEGMENT_SIZE) + segment[k];
}
size_t len = data_len%SEGMENT_SIZE;
// выравнивание на слово (опционально)
while (len >= 8) {
        h = fold(h ^ GET_U64(data, 0), 8);
        data+=8, len-=8;
}
// обработка остатка, по байтам (опционально)
while (len >= 1) {
        h = fold(h ^ GET_U8(data, 0), 1);
        data+=1, len-=1;
}
// модульная операция и миксер
return mixer(h);
```

При распределении вычислений между ядрами финальная функция `mixer()` выполняется со стороны хоста после слияния хешей от вычислительных ядер. Слияние по ядрам выполняется точно так же, через умножение на сдвиговые константы: `fold(h1, n) + h2`. Добавление `seed` можно выполнять после вычисления хэш ядрами, так же через слияние сегментов. В алгоритме мы можем переопределить оператор `+` и `^` под конкретную модульную операцию.

Методика тестирования

Методика тестирования включает множество тестов SMHasher, среди которых выделяются:

- `SpeedBulk` Тест скорости, показывает производительность в байтах входных данных на такт процессора.
- `SpeedSmall` Тест скорости на небольших объемах от 1 до 32 байта
- `Avalanche` / `BIC` Показательный тест насколько хорошо происходит замешивание бит, прохождение теста обеспечивается миксером.

- `Bias` (в avalanche-отчёте) Максимальное отклонение битов от 50%, не должно превышать 0.9%

Краткие описания тестов SMHasher:

Тест	Краткое описание
Avalanche	лавинный эффект (1 бит изменения → ~50% битов выхода)
Sparse	хеширование разреженных/почти пустых ключей
Permutation	тесты на перестановки ключей
Window	скользящее окно по большим ключам
Cyclic	периодические/циклические данные
TwoBytes	полный перебор всех 2-байтовых ключей
Text	тесты на текстовых строках/паттернах
Zeroes	ключи из нулей (начало/конец/все нули)
Seed	поведение при разных seed-ах
PerlinNoise	хеширование шума Перлина (плавные градиенты)
Diff	дифференциальные тесты (малые изменения)
DiffDist	распределение дифференциалов
BIC	Bit Independence Criterion (независимость битов)
MomentChi2	хи-квадрат по моментам распределения
Prng	хеш как генератор псевдослучайных чисел
BadSeeds	поиск плохих/слабых seed-ов

Наиболее значимые тесты и условия прохождения - Все тесты! Мы не используем в проектах хэш функции, которые не проходят все эти тесты. Заметим, что к хэш функциям для хэш-таблиц не предъявляется требование отсутствия коллизий. Наиболее чувстивтельные тесты к функции Avalanche minxer - `Avalanche` , `DiffDist` , `Sparse` . Ряд тестов выявляют коллизии в битах и перестановках. Некоторые тесты, такие как `TwoBytes` чувствительны к финализации хэш-функции на невыровненных данных.

`MomentChi2` единственный статистический тест в наборе, по которому выполнялось сравнение. Правильно подобранная константа генератора обеспечивает уровень прохождения `Great` и значения близкие к единице.

Функция генератора MWC с выбранным параметром должна проходить тест Crush TestU01 в качестве генератора ГПСЧ. И тест HWD, Hamming-Weight Dependencies, который обеспечивает тестирование паттернами на больших выборках порядка 10^{13}. Дополнительно должны запускаться тесты `Linear-complexity` по каждому биту и `Matrix-Runk` на существенно больших выборках.
Подобный подход в тестировании использован для LXM генератора.

Результаты работы

Алгоритмы некриптографического хэша:

- Модель обратимого хэш с skip/merge на базе MWC64-hash
- MWC64-hash с размером выхода 64 бита, состояние 64 бита.
- MWC128-hash с размером выходных данных 128 и 64 бита.
- MWC192-hash с размером выходных данных 192, 128 и 64 бита.
- Xoroshiro128-hash с длиной хеша 64 бита. Три варианта генератора с разным набором сдвиговых констант.
- xxHash64 с длиной хеша 64 бита, переносимая реализация с явной векторизацией.
- CRC64-hash Алгоритмы представлены с векторизацией и Folding x4 на операциях CLMUL - умножение без переносов, полиномиальное умножение в поле GF(2) для нескольких полиномов степени 64, 32. Представлены таблицы сдвиговых констант для CRC-32,CRC-32B, CRC-32/K (Koopman), CRC-32C (Castagnoli), CRC-32/Q, CRC-24/OPENPGP

Speed Test

Сервер Intel Xeon E5-2673 v4 Micro-architecture: Broadwell

hash	bytes/cycle	Скорость
mwc128	2.330	5098.79 MiB/sec @ 2295 MHz
mwc192	3.741	8188.38 MiB/sec @ 2295 MHz
xxh64	2.380	5209.48 MiB/sec @ 2295 MHz
xoroshiro	0.356	778.59 MiB/sec @ 2295 MHz

Тест `SpeedBulk` выполняется на блоках 256k и это может быть не показательно, когда хеши изготавливаются для коротких строк.

Тест `SpeedSmall` позволяет сравнить производительность на коротких строках, 8-16-32 байта (с выравниванием) и без выравнивания 1-32 байта,
На x86_64 MWC128 хэш выигрывает в производительности.

Заметим, тест xoroshiro выполняется для побайтовой операции, в то время как шаг алгоритмов слово или вектор 64 бит.

Одноплатный компьютер ARM AArch64 Cortex-A55:

hash	bytes/cycle	Скорость
xxh64	0.741	1086.17 MiB/sec @ 1536 MHz
mwc128	0.571	836.48 MiB/sec @ 1536 MHz
xoroshiro	0.175	256.95 MiB/sec @ 1536 MHz

Результаты теста SMHasher

- mwc128-64-hash-smhasher MWC128 с размером хэщ 64 бит
- mwc128-128-hash-smhasher MWC128 с размером хэш 128 бит
- xoroshiro128-hash-smhasher
- xxh64-hash-smhasher

Параметры генератора и миксера подлежат оптимизации, константы не рассчитывались, а заимствованы из других работ.

Дальнейшее развитие темы: оптимизация параметров, генератор с запаздыванием MWC64v2, генератор использующий вектор из генераторов MWC, и векторный алгоритм параллельной генерации хэш.

Результаты теста SMHasher3

SMHasher3 -- Существенно улучшено быстродействие, улучшено тестирование, более строгие политики по добавлению хэш-функций. Внесены существенные доработки тестов. Исторически следует отметить три проекта. Оригинальный Austin Appleby/SMHasher и два проекта базирующихся на оригинальном тесте Reini Urban/SMHasher и Frank J. T. Wojcik/SMHasher3. По качеству теста мы выбираем `SMHasher3` для отладки хэш-функций и представления сравнительных результатов.

Исходные коды для теста SMHasher3

- MWC128-hash 64-bit -- использован MUM-миксер
- signed MWC64 64-бит
- XXH64-hash 64-bit -- в алгоритм добавлен входной миксер `avalanche(seed)` для прохождения теста
- FNV1a-64 -- семейство алгоритмов, добавлен входной миксер на SEED и выходной миксер типа MUM. Ряд вариантов используют MUM раунд-миксер.
- CRC-64 -- алгоритмы CRC-64/NVME, CRC-64/XZ, CRC-64/ECMA и другие с разными полиномами степени 64. Реализованы оба варианта CRC-64: с прямым и отраженным порядком бит. В частности в ZFS для хеш-таблиц используется алгоритм CRC-64/ECMA с отраженным порядоком бит.
- Fletcher's checksum -- контрольная сумма на сложении по модулю простого числа. Fletcher4 - вариант алгоритма по модулю 2^{64} используется в ZFS на блоках 128K.

> Добавление входного и выходного миксера, дающего лавинный эффект смешивания бит, непременное условие прохождения теста. В практических задачах, таких как хэш таблицы, миксер можно исключить или заменить на облегченную версию. Условие прохождение теста - `Avalance` и `Bit Independence` требует, чтобы измерение одного бита влияло на все биты выходного значения.

Hash	Result	Tests	B/cycle	GiB/sec	SpeedSmall
MWC128-64	PASS	188/188	2.33	7.60	24.89
MWC128-128	PASS	188/188			31.70
xxh64-64	FAIL	179/188	6.19	20.17	33.07
FNV1a-64	PASS	186/186	0.39	1.28	48.58
FNV1a-64.mum	PASS	186/186	0.26	0.85	69.18
FNV1a-64.fast	PASS	188/188	6.81	22.21	28.34

Векторные расширения

Для обработки больших массивов данных (в том числе тензоров) перспективно развивать векторные варианты генератора: параллельное обновление нескольких 128-битных состояний с обменом carry между ними с применением перестановок/матричных миксов на уровне 4 слов (аналогично тому, как это делается в расширениях xoshiro256/512). Такие подходы позволяют

сохранить обратимость и возможность skip/merge, одновременно повышая пропускную способность на SIMD-архитектурах.

[1] David Blackman and Sebastiano Vigna. 2018. Scrambled Linear Pseudorandom Number Generators.
3 May 2018, 41 pages. arxiv:1805.01407 To appear in ACM Transactions on Mathematical Software.

[2] Austin Appleby. 2016. SMHasher. 8 Jan. 2016, https://github.com/aappleby/smhasher

[3] Richard P. Brent. 2004. Note on Marsaglia's Xorshift Random Number Generators.
Journal of Statistical Software, 11, 5, Aug., 1–5. coden:JSSOBK (https://doi.org/10.18637/jss.v011.i05)

[4] Guy L. Steele Jr. and Sebastiano Vigna. 2021. LXM: better splittable pseudorandom number generators (and almost as fast).
Proc. ACM Program. Lang. 5, OOPSLA, Article 148 (October 2021), 31 pages.
(https://doi.org/10.1145/3485525)

Сборка теста rurban/SMHasher

```
$ git clone https://github.com/rurban/smhasher.git
$ cd smhasher
$ git submodule update --init --recursive
$ cmake -B build
$ cmake --build build -j 16
; Если всё хорошо собралось, можно прогнать тесты для встроенной функции, их там много:
$ ./build/SMHasher prvhash64_64
```

Таблица тестов хэш-функций добавляется в `main.cpp`

```
{ xoroshiro_test, 64, 0xA8F45CF8, "xoroshiro",   "xoroshiro128_hash"},
{ mwc64_test,     64, 0xA8F45CF8, "mwc64",       "mwc64_hash"       },
{ mwc128_test,    64, 0x6ED613CD, "mwc128",      "mwc128-64_hash"   },
{ mwc192_test,   128, 0xC1D62836, "mwc192",      "mwc192-128_hash"  },
{ xxh64_test,     64, 0xA8F45CF8, "xxh64",       "xxh64_hash"       },
```

```
$ ./build/SMHasher mwc128
$ ./build/SMHasher mwc192
$ ./build/SMHasher xxh64
$ ./build/SMHasher xoroshiro
```

Сборка теста fwojcik/SMHasher3

```
$ git clone https://gitlab.com/fwojcik/smhasher3
$ cd smhasher3
$ cmake -B build
$ cmake --build build -j 16
```

Хэш-функии регистриуюся статически, вносить изменения в исходный код теста не требуется. Просто скопирвать хэши в папку `hashes/` и добавить имя файла в список компиляции `hashes/Hashsrc.cmake`.

```
$ ./build/SMHasher "mwc128-64"
$ ./build/SMHasher "mwc192-128"
$ ./build/SMHasher "xxh64-64"
$ ./build/SMHasher "xoroshiro-64"
```